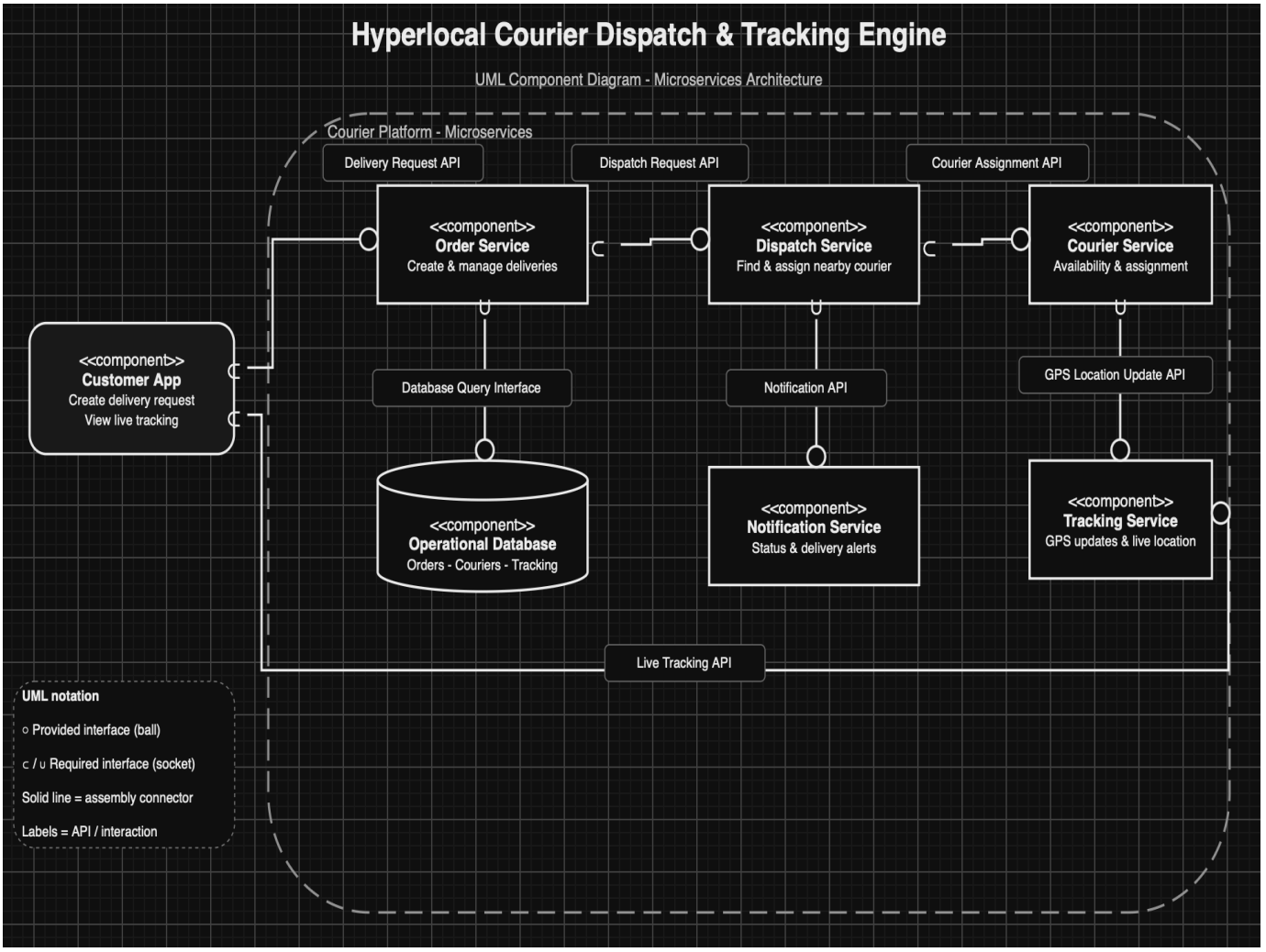
Lab 3: Component Modelling & Architectural Pattern Selection

Title: Hyperlocal Courier Dispatch & Tracking Engine
SRN: PES2UG24CS391
Selected Architecture: Microservices Architecture

Component Diagram



Architecture Selection

We chose **Microservices Architecture** for the **Hyperlocal Courier Dispatch & Tracking Engine**. In this architecture, the system is divided into multiple independent services, where each service is responsible for a specific function. The major components of our system are

the **Customer App, Order Service, Dispatch Service, Courier Service, Tracking Service, Notification Service, and Operational Database**.

The Customer App allows users to create delivery requests and view live tracking information. The Order Service manages delivery orders, while the Dispatch Service identifies and assigns a suitable nearby courier. The Courier Service manages courier availability and assignments. The Tracking Service handles GPS location updates and provides real-time delivery tracking. The Notification Service sends delivery status updates and alerts to users. The Operational Database stores information related to orders, couriers, and tracking.

Reasons for Choosing Microservices Architecture

1. Independent Scalability:
A hyperlocal courier system can receive a large number of delivery requests, courier-location updates, and tracking requests at the same time. Microservices allow heavily used services such as the **Tracking Service** or **Dispatch Service**to be scaled independently without increasing the resources of the entire application. This makes the system more suitable for handling increased demand.

2. Modularity and Fault Isolation:
Each major function of the courier platform is implemented as an independent service. For example, if the Notification Service temporarily fails, order creation, courier assignment, and live tracking can continue to operate. This improves system reliability and makes individual services easier to maintain, update, and deploy.

Security Advantage

Microservices architecture provides better **service isolation and controlled access between components**. Each service can expose only the APIs required by other services. For example, the Order Service communicates with the Dispatch Service through the **Dispatch Request API**, while the Courier and Tracking services communicate through the **GPS Location Update API**. Authentication and authorization can be applied to these APIs so that unauthorized users or services cannot directly access sensitive courier, order, or location information.

Performance Benefit

Microservices architecture improves performance because different services can process operations independently and concurrently. For example, courier assignment, GPS tracking, database operations, and notifications do not need to be processed as one large sequential operation. Services receiving heavy traffic, particularly the Tracking Service during real-time

GPS updates, can also be scaled independently. This helps reduce response time and allows the system to support a larger number of customers and couriers efficiently.

Conclusion

Therefore, **Microservices Architecture** is suitable for the Hyperlocal Courier Dispatch & Tracking Engine because it provides **independent scalability, modularity, fault isolation, improved security, and better performance**. It also allows individual services to be maintained and expanded independently as the courier platform grows.